INFO506 TP No1: Minishell

November 3, 2025



Le but de ce TP est de construire **un "Mini" Shell**. Qui va permettre de se déplacer dans le système de fichier et d'exécuter des commandes.

Pour réaliser cette application vous devrez maîtriser les appels systèmes stat, fork, exec, dup2 ainsi qu'un certain nombre de concepts: **processus, fichier, descripteur de fichier etc ...**

Consignes:

- Vous rendrez un fichier unique: **minishell.c**
- Vous répondrez aux questions dans une en-tête au début de votre fichier en indiquant le label de la question Q1, Q2, ...
- **Votre fichier doit compiler !**
- **Votre fichier ne doit pas provoquer d'erreur de segmentation.**
- Compilez votre fichier source avec :

  gcc –std=c99 -Wall -Wextra -Werror -o minishell ./minishell.c
- Utilisez au maximum le manuel de linux (commande man) et les fichiers d'exemples du Tp
- **Vous devez traiter les erreurs provoquées par les appels systèmes**. Pour cela vous utiliserez le document pdf présent dans le répertoire.

1 Minishell

1.1 Boucle d'interprétation des commandes

Le Shell (coquille) permet d'accéder aux fonctionnalités du **Kernel**(noyau) par l'intermédiaire de commandes (ls,cd,...).

On peut ainsi manipuler le système de fichier, créer de nouveaux processus, contrôler l'exécution des processus existants etc ...

La structure de base d'un shell est une boucle d'interprétation de commandes **REPL Read Eval Print Loop**.

Ce qui donnerai en simplifiant le pseudo-code suivant:

```
// Boucle d'interpretation des commandes REPL
 while(1){
                /* READ */
                // 1- Recuperation de la ligne de commande
                fgets(cmd,...);

                // 2- Recuperarion des elements de la commande
                get_cmd_args(...);

                /* EVAL */
                // 3- Traitement de la commande
                if(cmd = "exit"){
                    ...
                }
                else if(cmd = "pwd")){
                    ...
                }
                else if(cmd = "ls"){
                    ...
                }
                ...
}     /* LOOP */
```

1.2 Récupérer la ligne de commande

La fonction **fgets** permet de récupérer une chaîne de caractère depuis un flux ouvert sur un fichier.

```
char *fgets(char *s, int size, FILE *stream);
```

Les flux d'entrées-sorties standard sont déjà définis dans **stdio.h**.

```
FILE *stdin ;
FILE *stdout ;
FILE *stderr ;
```

Vous trouverez un exemple dans **EX0_fgets.c**.

Question1:

- ❖ Quelle est la différence entre la notion de FILE* et la notion de descripteur de fichier manipulé par open, read, write par exemple

1.3 Récupérer les arguments de la ligne de commande

Pour récupérer la commande et ses arguments créez la fonction get_cmd_args:

```
/*
cmdline la ligne de commande
cmdargs les arguments de la commande option fichier etc

Exemple :
cmdline="grep toto fic.txt"

cmdargs contiendra les valeurs
{ "grep", "toto", "fic.txt"}

*/
int get_cmd_args(char *cmdline, char **cmdargs);
```

La fonction **strok** de la LibC permet de découper une chaîne en fonction d'un caractère délimiteur.

```
char *strtok(char *str, const char *delim);
```

Attention cette fonction marche de façon itérative.

Vous pouvez regarder l'exemple **EX1_strtok.c** pour comprendre son fonctionnement.

1.4 Traiter la commande

Plusieurs cas de figure:

1. commande interne implémenté dans le shell: on exécute le code nécessaire
2. commande externe: provoque un fork et un exec
3. commande externe avec redirection (|,>,») : reconnaître la redirection et traiter les arguments après la redirection fichier, binaire ...

Pour comparer la commande avec les commandes implémentées dans le Shell vous utiliserez la fonction strcmp:

```
int strcmp(const char *s1, const char *s2);
```

strcmp retourne :

- < 0 si le dernier char non indique est inférieur
- 0 si tous les char sont identique
- > 0 si le dernier char diffèrent est supérieur

Vous pouvez regarder l'exemple **EX2_strcomp.c** pour comprendre son fonctionnement.

2 les commandes interprétées par le shell

Dans cette partie vous allez utiliser des appels système, vous devez absolument être capable de traiter les cas d'erreurs possibles !

2.1 quit/exit

Pour quitter votre shell vous utiliserez deux commandes :

quit qui va interrompre la boucle d'interprétation et donc effectuer le return de votre fonction main.

exit qui va invoquer l'appel système sys_exit au travers de la fonction exit.

On utilise habituellement les macros EXIT_SUCCESS, EXIT_FAILURE contenue dans l'entête stdlib.h. pour indiquer les conditions de sortie (au lieu de 0 ou 1). Testez le résultat de la sortie de votre minishell en affichant la variable $? .

```
> echo $?
```

Question2:

- ❖ Quelle est la différence entre ces deux méthodes. Que se passe t-il au niveau du système d'exploitation ?

2.2 pwd

La commande pwd permet d'afficher le chemin absolue du répertoire de travail du processus.

Pour obtenir cette information vous utiliserez la fonction :

```
char *getwd(char *buf);
```

Question3:

- ❖ Pourquoi cette notion de répertoire de travail est-elle importante ?

2.3 ls

Dans un premier temps la commande ls doit uniquement afficher le nom des fichiers du répertoire de travaille.

Pour cela vous devez utilisez les fonctions:

```
/**/
DIR *opendir(const char *name);
```

Retourne une structure DIR qui est un flux de lecture sur le contenu du répertoire. On exploite successivement les entrées avec la fonction readdir qui retourne NULL quand il n'y a plus d'entrées dans le répertoire.

```
struct dirent *readdir(DIR *dirp);
```

```
struct dirent {
               ino_t          d_ino;       /* Inode number */
               off_t          d_off;       /* Not an offset; see below
                   */
               unsigned short d_reclen;    /* Length of this record */
               unsigned char  d_type;      /* Type of file; not
                  supported
                                              by all filesystem types
                                                 */
               char           d_name[256]; /* Null-terminated filename
                   */
           };
```

Vous pouvez regarder l'exemple **EX3_readdir.c**.

2.4 lsl

Il s'agit d'une version étendue de ls qui permet d'avoir tout une série d'informations supplémentaires.

Pour cela vous utiliserez la fonction **stat** qui renseigne la structure stat passé en paramètre. La structure stat est décrie ci-dessous.

```
int stat(const char *pathname, struct stat *statbuf);
```

Vous pouvez regarder l'exemple **EX4_stat.c**.

```
struct stat {
               dev_t     st_dev;         /* ID of device containing
                  file */
               ino_t     st_ino;         /* Inode number */
               mode_t    st_mode;        /* File type and mode */
               nlink_t   st_nlink;       /* Number of hard links */
               uid_t     st_uid;         /* User ID of owner */
               gid_t     st_gid;         /* Group ID of owner */
               dev_t     st_rdev;        /* Device ID (if special file
                  ) */
               off_t     st_size;        /* Total size, in bytes */
               blksize_t st_blksize;     /* Block size for filesystem
                  I/O */
               blkcnt_t  st_blocks;      /* Number of 512B blocks
                  allocated */
               struct timespec st_atim;  /* Time of last access */
               struct timespec st_mtim;  /* Time of last modification
                  */
               struct timespec st_ctim;  /* Time of last status change
                   */

           };
```

Vous devez afficher pour chaque entrée :

1. les protections U G O (User Group Other)
2. la taille du fichier
3. le nombre de bloc 512o le no d'innode
4. le nom du fichier

La protection s'obtient en appliquant un filtre sur le champs st_mode. Ces filtres sont défini dans stat.h.

```
#define S_IRUSR 0000400 /*R for owner*/
#define S_IWUSR 0000200 /*W for owner*/
#define S_IXUSR 0000100 /*X for owner*/
#define S_IRGRP 0000040 /*R for group*/
#define S_IWGRP 0000020 /*W for group*/
#define S_IXGRP 0000010 /*X for group*/
#define S_IROTH 0000004 /*R for other*/
#define S_IWOTH 0000002 /*W for other*/
#define S_IXOTH 0000001 /*X for other*/
```

Par exemple :

```
printf((filestat.st_mode & S_IROTH) ? "r" : "-" );
```

Permet d'afficher si le fichier est en lecture pour les autres.

2.5 cd

La commande cd permet de changer de répertoire de travail. Pour cela on utilise la fonction:

```
#include <unistd.h>

int chdir(const char *path);
```

Vous pouvez conserver le répertoire courant avant de changer de répertoire pour implémenter le retour sur le dernier répertoire: cd -

3 Exécuter un programme

3.1 exécuter programme simple

Le seconde fonctionnalité d'un shell est de pouvoir exécuter des programmes. Pour cela on utilise **l'appel système exec** qui permet de remplacer le code du processus appelant par celui d'un nouveau programme.

```
int execl(const char *pathname, const char *arg, ..., (char*) NULL);
int execlp(const char *file, const char *arg, ...,(char*) NULL );
int execle(const char *pathname, const char *arg, ..., (char*) NULL,
   char *const envp[]);
int execv(const char *pathname, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execvpe(const char *file, char *const argv[],char *const envp[]);
```

Cette série de fonction comporte les lettres :

- l transmet les arguments sous forme de liste
- v sous forme char *argv[]
- e l'environnement du processus est transmis sous le forme char* envp[]
- p permet de chercher les programme dans les répertoires de la variable PATH

Vous pouvez regarder le code de **./EX5_exec.c**

Vous ne devez pas remplacer votre minishell par le code du nouveau programme, sinon votre boucle d'interprétation de commande sera interrompue!

Votre nouvelle commande doit être exécutée par un processus fils au moyen d'un fork. Le shell doit attendre de lire le code de retour de ce processus.

3.2 Redirection vers un fichier

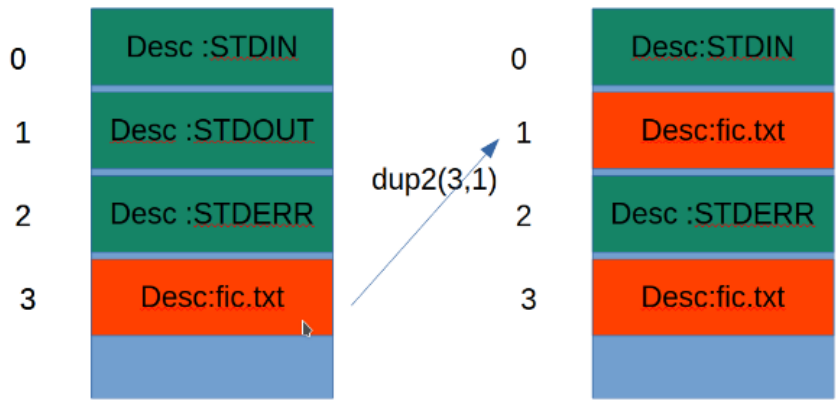
Le symbole de redirection > permet de rediriger le résultat d'une commande exécutée dans le shell vers un fichier : date > fic . txt

Pour réaliser cette fonctionnalité vous utiliserez les fonctions:

```
int open(const char *pathname, int flags);
int dup2(int oldfd, int newfd);
```

L'appel système dup2 permet de remplacer un descripteur de fichier par un autre dans la table des descripteur.

Si l'on veut rediriger la sortie standard on doit recopier le descripteur du nouveau fichier à la position 1 pour remplacer celui de la sortie standard.



Toute les écritures vers stdin se font maintenant vers fic.txt

Regardez l'exemple contenu dans le fichier **EX6_dup2.c**

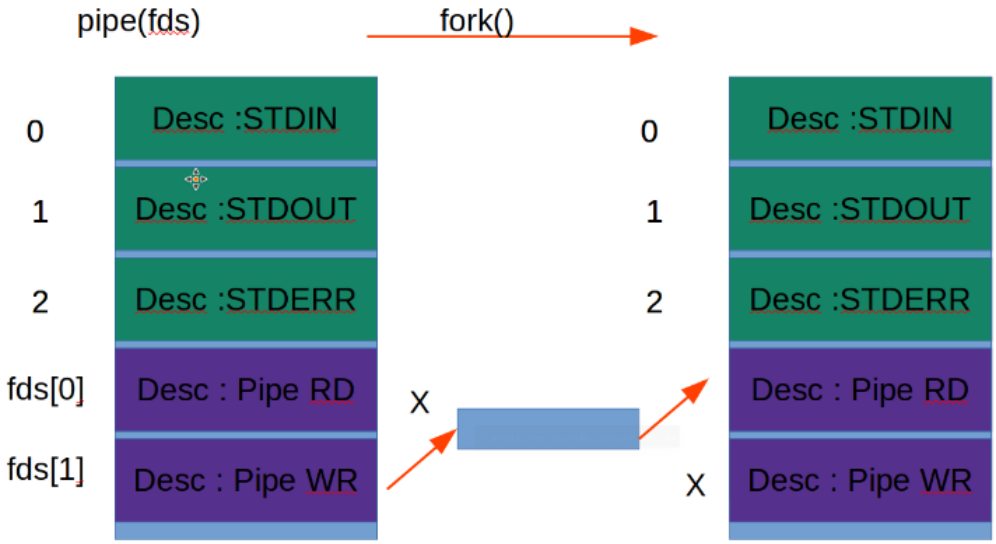
Vous devez traiter la présence du symbole de redirection dans le tableau des arguments cmdargs.

3.3 Redirections vers l'entrée standard d'un autre programme

Les shell tels que bash permettent de chaîner les commandes **grâce au caractère |**. La sortie de la première commande est envoyé à la seconde.

Par exemple : ls -l | wc -l

Pour implémenter cette fonctionnalité on utilise un type de fichier particulier le pipe qui va permettre de relier les deux processus. **Ce fichier possède deux descripteurs de fichier un pour l'écriture l'autre pour la lecture.**



Regardez l'exemple contenu dans le fichier **EX7_dup2.c**

Dans notre cas le premier processus devra écrire sur le pipe au lieu d'écrire sur stdout (terminal), le deuxième devra lire sur le pipe au lieu de lire sur stdin.